

17

Input Validations in Forms

In the previous chapter, we discussed protecting your pages and creating a login form and a registration form. However, we didn't write any validations that will prevent users from entering invalid inputs.

This chapter will add validations to our forms, which will improve the user experience whenever users use our forms. So let's start.

We will cover the following topics:

- Installing an input validation library
- Using validators in forms

Technical requirements

You need Visual Studio Code to complete this chapter.

Here is the link to the finished repository of this chapter: <https://github.com/PacktPublishing/ASP.NET-Core-and-Vue.js/tree/master/Chapter17/>.

Installing an input validation library

A user-friendly validation informs the user why the field is not valid while typing the input. We can build it from scratch by writing a validator in every field of the form and making sure that the validators are reactive. The implementation is doable, but it will require a lot of time, and the cleanliness of the code will depend on the developer.

So why not use a library that validates us? Fortunately, there are several Vue.js libraries for validation, and in this section, we will use one of the top libraries.

So let's install an input validation library:

```
npm i vuelidate
```

The `vuelidate` library is a simple lightweight model-based validation that we can use for our Vue application.

Now, let's create a JavaScript file called `vuelidate.js` in the `src/plugins` folder and apply the following code:

```
import Vue from "vue";
import Vuelidate from "vuelidate";
Vue.use(Vuelidate);
```

The preceding code imports `Vue` and `Vuelidate` and then passes `Vuelidate` to the `Vue use` method.

Next, we update `main.js` with the following code:

```
import "../plugins/vuelidate";
```

Let's bring in `Vuelidate` from the file we just created. That's all.

Next, we create a folder called `validators`, and a JavaScript file called `index.js` inside the folder. It should be like `validators/index.js`:

```
import { required, email, minLength } from "vuelidate/lib/validators";

export default {
  login: {
    email: { required, email },
```

```
password: { required, minLength: minLength(8) },  
},  
};
```

We define our validators in a separate file for cohesiveness. We have a login object that has an email. The email is required and must be in an email format only. The login object also has a password object, which is also required and has a minimum of eight characters.

Simple and easy, right? Now let's use some validators.

Using validators in forms

Using validators in forms is easy since we are using an npm library for validating forms, and we are not re-inventing the wheel.

Now, let's update the `Login.vue` page in the `auth/views` folder:

```
import validators from "@/validators";
```

Let's import validators, which we've created, and then use validators inside the `computed` block like so:

```
computed: {  
  emailErrors() {  
    const errors = [];  
    if (!this.$v.login.email.$dirty) return errors;  
    !this.$v.login.email.email && errors.push("Must be  
    valid e-mail");  
    !this.$v.login.email.required && errors.push("E-mail  
    is required");  
  
    return errors;  
  },  
  passwordErrors() {  
    const errors = [];  
    if (!this.$v.login.password.$dirty) return errors;  
    !this.$v.login.password.required &&  
    errors.push("Password is required");  
    !this.$v.login.password.minLength &&
```

```
        errors.push("Minimum characters is 8");  
  
        return errors;  
    },  
    },  
    validations: {  
        login: validators.login,  
    },  
}
```

We have a computed method for email errors. `emailErrors` initializes an array of errors. It checks whether the email field is dirty, meaning the value of the field has changed. It also checks whether the email field is not valid. `emailErrors` also checks whether the email is empty.

The pattern is also the same as in the `passwordErrors` computed method. The dollar sign means that the properties are reactive to any changes.

And then, we declare a `validations` block of an object with the `login` property and assign `validators.login` to it.

The next step is to update the `v-text-field` component of the login field with the following code:

```
<v-text-field  
  label="E-mail"  
  v-model="login.email"  
  @input="$v.login.email.$touch()"  
  @blur="$v.login.email.$touch()"  
  :error-messages="emailErrors"  
></v-text-field>
```

We add the error checkers in the preceding code. The error checkers are reactive; they run as soon as we type something and lose focus on the fields. `:error-messages` is a directive from Vuetify that we can use to assign the `emailErrors` computed method. That's neat isn't it?

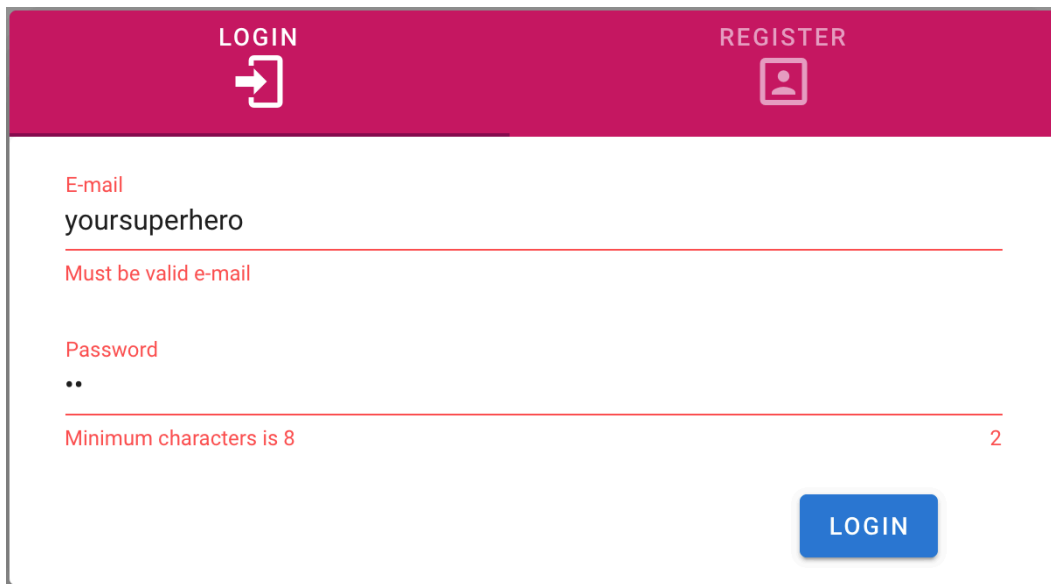
Let's also update the `v-text-field` component for the login's password like so:

```
<v-text-field  
  type="password"  
  label="Password"
```

```
hint="At least 8 characters"
counter
v-model="login.password"
@input="$v.login.password.$touch()"
@blur="$v.login.password.$touch()"
:error-messages="passwordErrors"
></v-text-field>
```

The preceding code attaches the validators to `v-text-field` for the login password. We are using `:error-messages` with the value `passwordErrors`.

Now, run the application and then type `yoursuperhero` in the **Login** form without finishing the email, and then add two letters to the **Password** field:



The screenshot shows a login form with a pink header. On the left, there is a 'LOGIN' button with a right-pointing arrow icon. On the right, there is a 'REGISTER' button with a person icon. Below the header, there are two input fields. The first field is labeled 'E-mail' in red and contains the text 'yoursuperhero'. Below this field, there is a red error message: 'Must be valid e-mail'. The second field is labeled 'Password' in red and contains two dots. Below this field, there is a red error message: 'Minimum characters is 8'. A blue 'LOGIN' button is located at the bottom right of the form.

Figure 17.1 – Not valid

Figure 17.1 shows that `yoursuperhero` without `@domain.com` is not valid, as well as the fact that the two letters that are supposed to be eight letters/numbers are not valid. The field is turning from black to red and red texts below the fields show that the validators are working.

That's good. We now have a proof of concept that the input validation library is working. Let's add more field validations to other forms.

The first step is to update the `index.js` file of the `validators` folder:

```
import {
  required,
  email,
  minLength,
  maxLength,
} from "vuelidate/lib/validators";
```

We add `maxLength` to the existing imports from `vuelidate/lib/validators`.

Then we update the exported object by adding `city`, `country`, and `about`:

```
export default {
  login: {
    email: { required, email },
    password: { required, minLength: minLength(8) },
  },
  city: {
    required,
    maxLength: maxLength(90),
  },
  country: {
    required,
  },
  about: {
    required,
  },
};
```

Most of the objects added are required fields in the following form that we are going to fix.

Now, update the `AddTourListForm.vue` component of the `components` folder with the following code:

```
import validators from "@/validators";
```

Let's import validators and then add a computed object with error checker methods and the validations object like so:

```
computed: {
  cityErrors() {
    const errors = [];
    if (!this.$v.bodyRequest.city.$dirty) return errors;
    !this.$v.bodyRequest.city.required &&
    errors.push("City is required");
    !this.$v.bodyRequest.city.maxLength &&
    errors.push("Max length is 90");
    return errors;
  },
  countryErrors() {
    const errors = [];
    if (!this.$v.bodyRequest.country.$dirty) return
    errors;
    !this.$v.bodyRequest.country.required &&
    errors.push("Country is required");
    // no need for max length because this is a dropdown
    with options
    return errors;
  },
  aboutErrors() {
    const errors = [];
    if (!this.$v.bodyRequest.about.$dirty) return errors;
    !this.$v.bodyRequest.about.required &&
    errors.push("About is required");
    return errors;
  },
},
validations: {
  bodyRequest: {
    // for brevity, please go to the github repo
  },
},
},
```

We have the `cityErrors` computed method, which adds an error if the `city` field is empty and the maximum length exceeded 90 characters. While the `country` field only needs a required validation because the maximum length wouldn't be needed in a drop-down list or autocomplete component, `aboutErrors`, on the other hand, only checks whether the field is empty.

Meanwhile, the `validations` object initializes a `bodyRequest` object with properties coming from the `validators/index.js` file.

Now, let's update the template syntax of the `AddTourListForm.vue` component by editing `v-text-field` of the `city` input:

```
<v-text-field
  label="City"
  v-model="bodyRequest.city"
  @input="$v.bodyRequest.city.$touch()"
  @blur="$v.bodyRequest.city.$touch()"
  :error-messages="cityErrors"
  required
></v-text-field>
```

We added `cityErrors` for the `:error-messages` directive. We updated the `@input` and `@blur` events.

Let's also edit the `v-autocomplete` component in the template syntax, like so:

```
<v-autocomplete
  :items="countryList"
  label="Country"
  v-model="bodyRequest.country"
  @input="$v.bodyRequest.country.$touch()"
  @blur="$v.bodyRequest.country.$touch()"
  :error-messages="countryErrors"
  required
></v-autocomplete>
```

The preceding code updates `@input`, `@blur`, and `:error-messages` with validators and error messages for country input.

Next, we update the field, actually, the text area, for the about input:

```
<v-textarea
  label="About "
  v-model="bodyRequest.about "
  @input="$v.bodyRequest.about.$touch()"
  @blur="$v.bodyRequest.about.$touch()"
  :error-messages="aboutErrors"
  required
></v-textarea>
```

In the preceding code, let's use the computed errors and the error messages in the `v-textarea` component. The preceding code also attaches the validation to `textarea`.

And now, rerun the application.

Write a sentence with 100 characters in the **City** field, pick a country, click on the **About** text area, and then click again in the **City** field. This is to create a sample entry to our form like so:

Create New Tour List

City
 Lorem Ipsum is simply dummy text o
 Max length is 90

Country
 Poland ▼

About
 About is required

*indicates required field

CLOSE SAVE

Figure 17.2 – New Tour List form validations

Figure 17.2 shows that the input validations in the **Create New Tour List** form are working. There is an error in the **City** field because the city's maximum characters are 90, while the **About** field is empty.

Activities

Let's test your skills in Vue.js by finishing up the Register Form's validation, the validation of `AddTourPackageForm.vue`, and the validation of `UpdateTourPackageForm.vue`.

The link to Vuelidate is <https://vuelidate.js.org/>. There is another excellent input validation library that you might want to check out. The library is called **VeeValidate** (<https://vee-validate.logaretm.com/v4>). VeeValidate is what I can recommend if you are planning to use Vue.js version 3. The library is easy to use and has developer-friendly APIs. The library is also well integrated with Yup validation, which is a JavaScript schema builder for validation.

So let's summarize what you have learned.

Summary

Let's quickly summarize what you have learned in this chapter to wrap it up. You have learned how to install Vuelidate and set it up in a Vue application. You have also learned how to use Vuelidate to create input validations in our forms, which provides better usability to our application's forms.

In the next chapter, you will see how important it is and, at the same time, how fun it is, to write tests in ASP.NET Core apps.